Why parallel processing

Power density, clock distribution, limited ILP, and memory stalls constrain a faster single core. Parallel hardware aims to increase completed work or shorten a large problem's elapsed time.

A multicore puts cores on one die. A multiprocessor can span sockets. A cluster spans computers and a network. Communication latency, coherence, and failure boundaries differ. Every speedup needs an explicit workload and baseline.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Kinds of parallelism

Independent jobs can run concurrently without dividing one job. A parallel program instead distributes one problem among workers, subject to dependencies.

ILP overlaps operations in one instruction stream. Data parallelism applies similar work to many elements. Task parallelism distributes different functions or requests. These can coexist in one system.

Decide whether the goal is single-request latency or aggregate throughput before choosing a decomposition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.1 Power-aware design

Lower voltage and frequency can reduce dynamic energy per operation while reducing per-core speed. Several slower cores may improve aggregate work per watt only if software exposes useful parallel work.

Replicated services can continue after a node failure, but require detection and recovery. Core count alone does not provide availability. Scalable software must remain correct across counts, cache capacities, and topologies.

An efficiency claim needs voltage/frequency policy, workload, and system power boundaries.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.2 Amdahl's fixed-work model

A fixed workload takes baseline single-thread time T_0. Fraction f of that time parallelizes perfectly among N workers; the rest remains serial. With no added costs,

$$T_N = T_0(1-f) + T_0 f/N, \qquad S_N = T_0/T_N = 1/(1-f+f/N).$$

Efficiency is $E_N = S_N/N$, distinct from occupancy. For $N = 8$ and $S_8 = 4.70588$, $E_8 = 0.588235 = 58.8235\%$.

Amdahl, strong/weak scaling, and workload bounds

Fixed work

T_N = T_seq + T_par/N

T_0 = T_seq + T_par

Speedup limit

S_N = T_0/T_N

S_∞ = 1/(1−f)

Required fraction

f ≥ (1−1/S)/(1−1/N)

S ≤ N; equality needs f=1, zero added cost

N=100, target S=90: f≥890/891≈0.998878; serial≤1/891≈0.1122%.

10 scalar + 1000 matrix additions, N=100: T=10+10=20.

Grow matrix work to 10000 additions: T=10+100=110, S=10010/110≈91.

Strong scaling fixes the problem; weak scaling grows it with workers. Their baselines and conclusions differ.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.2 Amdahl's fixed-work model (continued)

$$S_\infty = 1/(1-f).$$

The asymptote holds fixed workload, baseline time fractions, and costs. It is not a measured machine result.

Inverting a speedup target

From $1/S = 1 - f(1 - 1/N)$, a target $1 < S < N$ requires

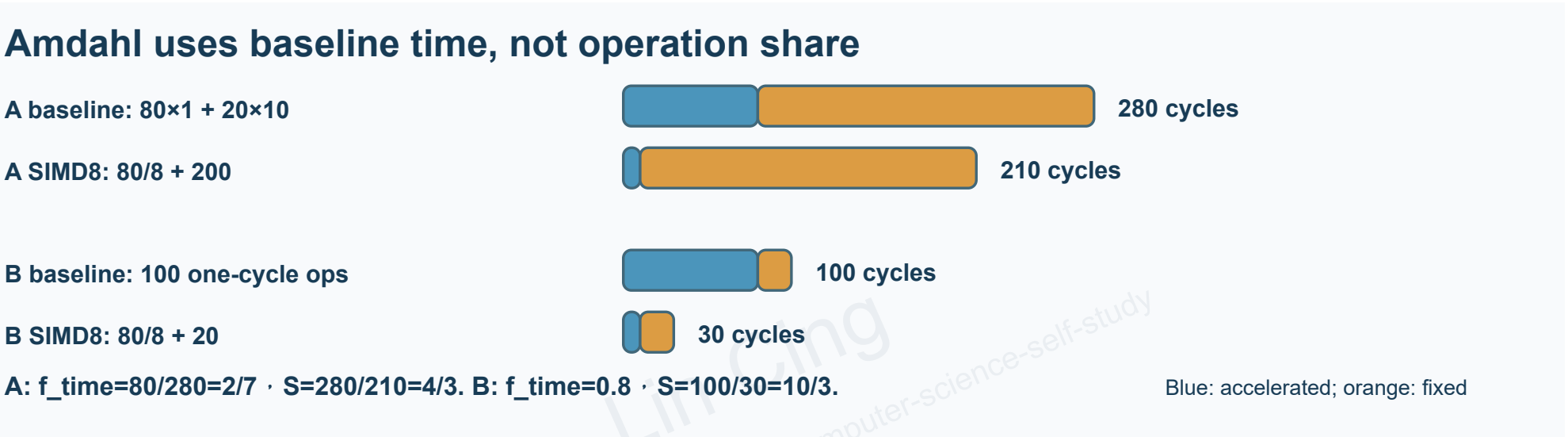
$$f \geq \frac{N(S-1)}{S(N-1)}.$$

N=100, S=90 requires f≥8900/8910=0.998878 and serial fraction≤0.1122%, not merely 90% parallel work.

$$T_{N,observed} = T_N + T_{critical} + T_{sync} + T_{contention}.$$

Launch, imbalance, synchronization, and shared bandwidth are excluded from the ideal model. Verify which costs are negligible before attributing a measured gap to one cause.

Operation fractions versus time fractions



Eighty one-cycle operations vectorize; twenty ten-cycle operations do not. Baseline is 80+200=280 cycles. Eightfold acceleration gives 10+200=210 cycles, speedup 4/3. The accelerated baseline time fraction is 80/280=2/7.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Operation fractions versus time fractions (continued)

Model	Accelerated baseline time	Other time	Time after eightfold acceleration	Speedup
A, unequal costs	80	200	210	1.3333
B, all 100 operations cost one cycle	80	20	30	3.3333

The same 80% operation share has different time shares. Substitute 0.8 into Amdahl only under equal costs or a measured 80% baseline-time fraction.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Strong scaling, weak scaling, and efficiency

Strong scaling fixes W and increases P:

$$S_{strong}(P) = T_1(W)/T_P(W), \qquad E(P) = S_{strong}(P)/P.$$

Weak scaling grows work to PW while holding per-worker work fixed:

$$E_{weak}(P) = T_1(W)/T_P(PW).$$

Ideal weak efficiency is 1. This time ratio compares different total problem sizes and is not fixed-work speedup; do not divide it by P again. Only assuming linear single-core cost, $T_1(PW) = PT_1(W)$, permits

$$S_{scaled}(P) = T_1(PW)/T_P(PW) = PE_{weak}(P).$$

Without a larger-problem single-core time or that linear-cost assumption, the same-size speedup is unknown. Fixed total memory M gives about M/P per worker under strong scaling; growing total memory to PM holds about M per worker under weak scaling.

Two complete addition cases

Each addition costs t; ten additions remain serial.

Strong scaling, weak scaling, and efficiency (continued)

Problem	P=10	P=100
Small, 10 serial+100 parallel	T=20 t, S=110/20=5.5	T=11 t, S=110/11=10
Large, 10 serial+10000 parallel	T=1010 t, S=10010/1010=9.91	T=110 t, S=10010/110=91

The small fixed workload reaches the serial floor quickly. More parallel work dilutes the same serial cost in the large workload. Amdahl and weak scaling are consistent because work and baseline change together. Record problem size, per-core work, workers, baseline, bytes communicated, memory footprint, clock/power policy, and wall time.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Load balance and the critical path

Let total parallel work be W, heavy-worker work Wmax, workers P, and fixed overhead C. If other workers evenly divide the remainder,

$$T_{parallel} = \max(W_{max}, (W - W_{max})/(P-1)), \qquad T_{total} = T_{parallel} + C.$$

Complete example

W=10000, C=10, P=100. A heavy worker with 2% receives 200 units; others each get 9800/99=98.99.

$$T = \max(200, 9800/99) + 10 = 210, \qquad S = 10010/210 = 47.67, \quad E = 47.7\%.$$

At 5% heavy work,

$$T = \max(500, 9500/99) + 10 = 510, \qquad S = 10010/510 = 19.63, \quad E = 19.6\%.$$

Average worker load does not predict completion. The slowest worker or dependency chain controls it. Work stealing adds queue/atomic metadata and migration-related cache/network traffic; tiny tasks may cost more to coordinate than the balance gain saves.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.3 Shared-memory multiprocessors

A shared physical address space permits communication with loads and stores. Coherence maintains the single-location copy/order contract.

UMA approximates equal memory access costs from different processors. NUMA gives local and remote memory different latency and bandwidth. It still provides a shared address space. Page placement and task affinity therefore matter.

Directory protocols can scale coherence beyond a small broadcast snooping domain.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Shared-memory synchronization

Atomicity prevents externally visible intermediate RMW state. Ordering specifies which operations precede others. Visibility determines the value a consumer sees once those orderings exist. Coherence supplies same-location serialization; cross-location publication needs synchronization from the memory model.

Shared-data publication: atomicity, ordering, visibility



Release orders data before ready. An acquire that reads that release value synchronizes with it, so the reader's later data load sees 42.

Separate relaxed variant: both objects atomic; ready=1 can precede a data load returning 0.

Conflicting unprotected non-atomic accesses race. Acquire/release orders only accesses covered by its contract.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Publication example

```
int data = 0;                  // shared object
atomic_int ready = 0;
// Writer
data = 42;
atomic_store_explicit(&ready, 1, memory_order_release);
// Reader
while (!atomic_load_explicit(&ready, memory_order_acquire))
    yield();
int observed = data;
```

Only an acquire that actually reads the writer's released 1 synchronizes with that store. Program order on each side then forms a happens-before chain from data=42 to its load, giving observed=42.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Shared-memory synchronization (continued)

For a defined counterexample, make both data and ready atomic and use memory_order_relaxed for all four accesses. The writer stores data=42 before ready=1; a reader observing ready=1 may still load data=0 because no cross-object synchronizes-with relation exists. Coherence orders each atomic object separately. Plain non-atomic conflicting accesses without other synchronization instead cause a C/C++ data race and undefined behavior; their possible results cannot be restricted to 0 and 42.

Synchronization covers only the conflicting accesses it orders. A third thread writing outside that protection still races; an atomic flag does not make a multi-location transaction atomic. Spinlocks consume interconnect/power under contention. Blocking mutexes pay OS/context costs. Barriers wait for all participants; a tree can reduce depth to O(logP) while retaining per-node synchronization/wakeup costs.

7.4 Clusters and message passing

Cluster nodes have separate physical address spaces and typically separate OS instances. Sends, receives, RPCs, and queues transfer information explicitly.

Encoding, buffering or copying, routing, receiving, and any acknowledgment contribute cost. Isolation does not remove recovery logic. Specialized MPP networks and commodity cluster networks make different price, latency, and bandwidth tradeoffs.

A distributed shared-memory abstraction still needs communication beneath it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

7.5 Hardware multithreading

Hardware threads retain separate PCs, register state, and selected controls while sharing execution and memory resources.

Fine-grained multithreading selects among threads frequently, often every cycle. Coarse-grained switching responds to long stalls. SMT fills superscalar issue slots with ready work from different threads in the same cycle.

All can hide latency, but they add state and competition. Two threads/core do not guarantee twice the throughput, and a thread may take longer when sharing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.6 Hardware and programming classifications

SISD has one instruction and data stream. MIMD has independent instruction/data streams. SIMD broadcasts an operation over multiple elements. SPMD is a programming model in which threads execute the same program on different data.

SPMD can target MIMD clusters or SIMT GPUs. It is not a separate hardware width. SIMD suits regular contiguous operations, while divergent control and irregular dependencies reduce lane use.

Separate software decomposition from hardware execution before assigning a label.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Vector architecture

A vector ISA expresses operations over a vector length, while hardware executes through lanes, pipelines, and chaining. Vector length need not equal physical lane count. Fixed-width multimedia SIMD often requires explicit width-dependent blocks and tail handling.

Let each element require k operations, with L lanes each completing one element/cycle, frequency f, and useful utilization η.

$$throughput = kLf\eta \text{ operations/s}.$$

For lane initiation interval q cycles/element, divide by q. Alternatively, vector length W and issue rate r vectors/cycle give $kWrf$. Do not multiply both descriptions together and count work twice.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

7.7 GPU positioning

GPUs use many threads, warps, and high-bandwidth memory for aggregate throughput. CPUs prioritize fewer latency-sensitive streams. GPU scheduling hides latency with other ready work rather than making every thread's operation fast.

Transfers, launches, divergence, coalescing, occupancy, and synchronization determine end-to-end gains. Appendix A develops the execution and memory model.

Core-count ratios are not a scalar-program speedup formula.

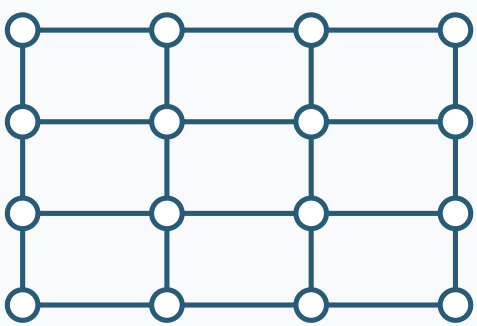
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.8 Multiprocessor network topologies

Represent endpoints and links as a graph. Count a full-duplex physical link once; state whether cut bandwidth counts one direction or both.

A four-by-four mesh

4×4 mesh: 24 local links, no wraps



Mesh metrics

horizontal = 4×(4−1) = 12

vertical = 4×(4−1) = 12

edges = 24; average degree = 48/16 = 3

corners degree 2; interior degree 4

diameter = (4−1)+(4−1) = 6

bisection links = 4

No opposite boundaries connect, so this is a mesh rather than a torus.

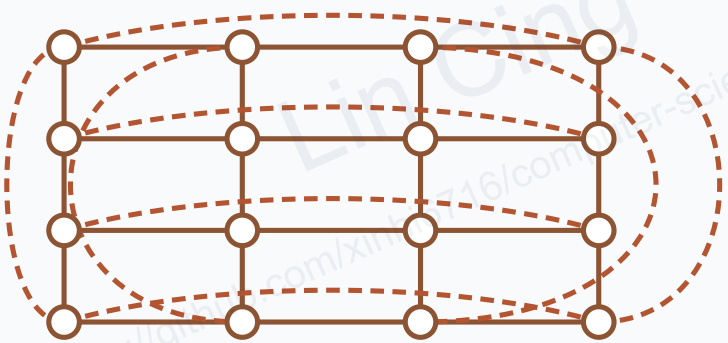
$$|E| = 4(4-1) + 4(4-1) = 12 + 12 = 24.$$

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Degree sum=48, average 3; corners have degree 2, edges 3, interior 4. The longest shortest path is 3 horizontal+3 vertical=6 hops. Splitting columns 0–1 from 2–3 crosses four horizontal links; the minimum balanced cut has four.

A four-by-four torus

4×4 torus: 24 local + 8 dashed wrap = 32 links



H0–H3: (0,r)↔(3,r). V0–V3: (c,0)↔(c,3). Dashed lines are all eight wraps.

Every row and column forms a four-node cycle. Former mesh corners have degree four in the torus.

Torus metrics

degree = 4 for every node

wrap links = 4 horizontal + 4 vertical

total edges = 24 + 8 = 32

diameter = floor(4/2)×2 = 4

bisection links = 8

7.8 Multiprocessor network topologies (continued)

Add four row wraps and four column wraps to the mesh:

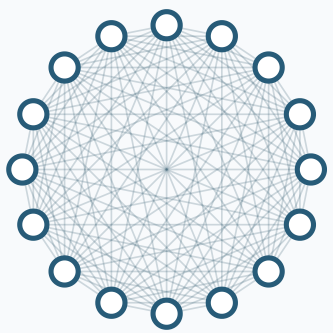
$$|E| = 24 + 4 + 4 = 32, \qquad degree = 4, \qquad diameter = 2 + 2 = 4.$$

Each dimension's shortest wrap distance is at most floor(4/2)=2. The same column cut crosses four local and four wrap links; minimum bisection is 8. Horizontal wraps Hr connect (0, r)↔(3, r), r=0..3. Vertical wraps Vc connect (c, 0)↔(c, 3), c=0..3. Omitting one of these eight changes the endpoint degrees; omitting all leaves a mesh.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

K16 and a central crossbar

K16: every pair of 16 endpoints has a direct link



K16 metrics

edges = N(N−1)/2 = 16×15/2 = 120

degree = 15

diameter = 1

bisection = (N/2)² = 8² = 64 links

One-way cut capacity = 64×link rate

K16 and a central crossbar use different endpoint and switch-resource models.

For full-duplex links, both cut directions total 2×64×one-way rate. State the counting convention.

$$|E| = 16(15)/2 = 120, \quad degree = 15, \quad diameter = 1.$$

Every balanced 8/8 cut crosses 64 links. At one-way link rate b, one-way bisection bandwidth is 64 b. A central N-port crossbar instead normally connects each endpoint to one port and has $O(N^2)$ internal crosspoints. Its arbitration, wires, endpoint degree, and port limits need a switch model.

7.8 Multiprocessor network topologies (continued)

Total link bandwidth sums link capacities; bisection uses the smallest balanced cut. Neither determines application latency by itself. Hotspots, routing, message size, and flow control matter.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.9 Multiprocessor benchmarks

Fix program, data set, correctness validation, compiler, processor count, and scaling rule. Record growing problem size explicitly for weak scaling.

Dense-linear-algebra benchmarks, independent-job rate tests, and shared-memory application suites stress different resources. Peak FLOP/s is not measured application performance. Communication, memory bandwidth, imbalance, synchronization, energy, and tails can dominate.

Report the scaling curve rather than one largest-machine number.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.10 Deriving Roofline

A phase performs F FLOPs and transfers Q bytes across a specified memory interface. Arithmetic intensity is I=F/Q. Sustained compute ceiling PC and interface bandwidth B imply

$$T_{compute} = F/P_C, \qquad T_{memory} = Q/B = F/(IB).$$

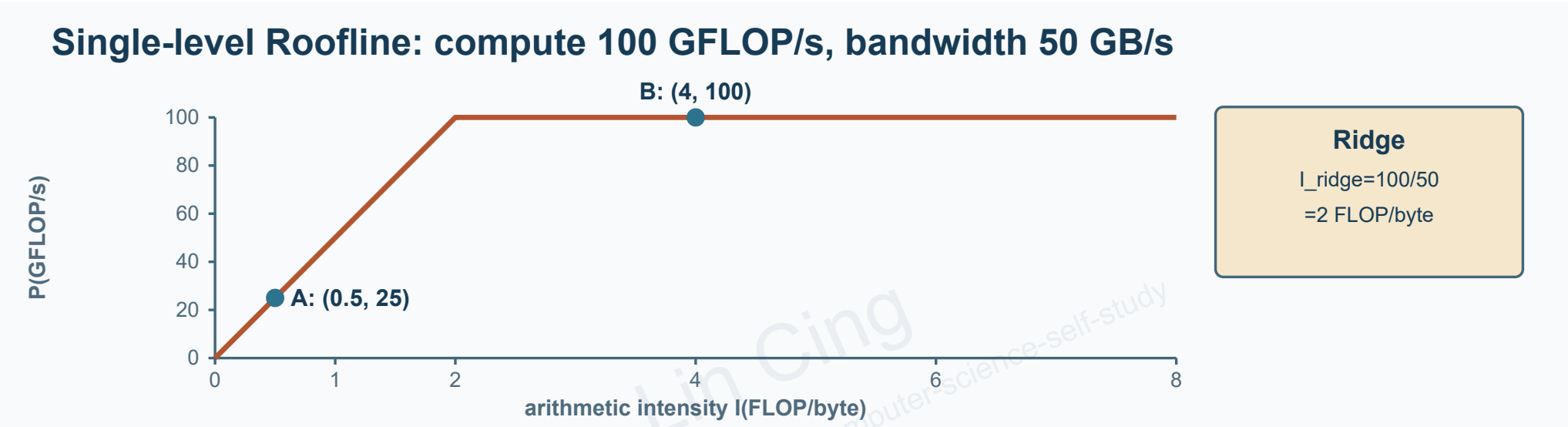
Even with complete overlap,

$$T \geq \max(F/P_C, F/(IB)), \qquad P = F/T \leq \min(P_C, IB).$$

Equating the two roofs gives $I_{ridge} = P_C/B$. Lower intensities are memory-limited; higher ones are compute-limited. The bound is not a promise that dependencies, mix, imbalance, and transaction efficiency permit reaching it.

Single-level working points

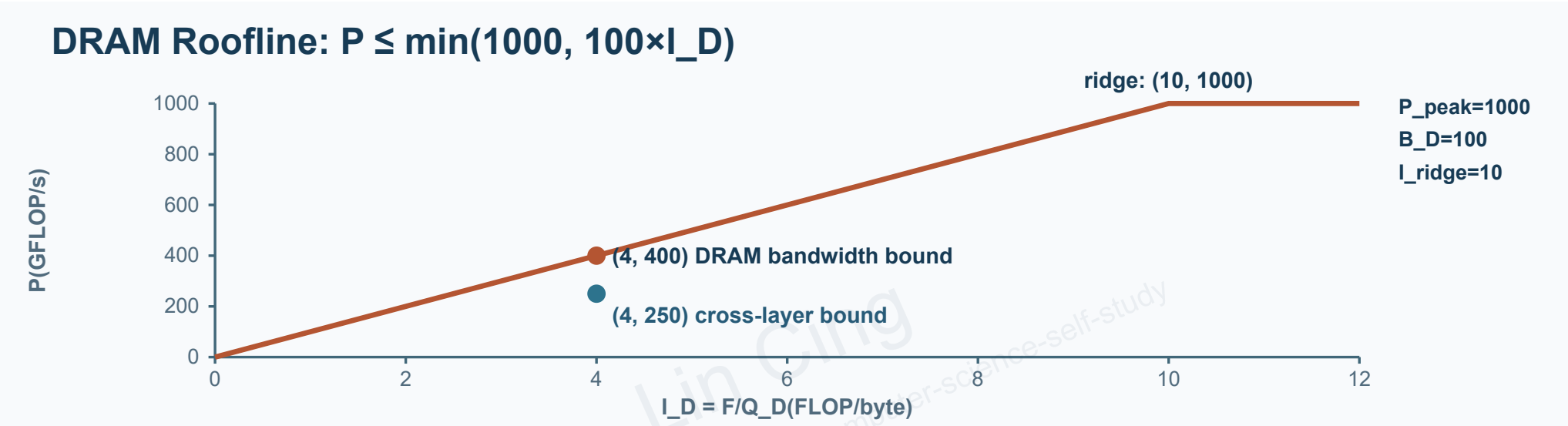
PC=100 GFLOP/s and B=50 GB/s give ridge 2 FLOP/byte. A at I=0.5 has memory roof 25 and bound 25 GFLOP/s. B at I=4 has memory roof 200 but compute caps it at 100 GFLOP/s.



One phase across two memory interfaces

Fix $F = 10^9$ FLOPs and $P_C = 1000$ GFLOP/s. DRAM transfers $Q_D = 0.25 \times 10^9$ bytes at $B_D = 100$ GB/s; shared memory transfers $Q_S = 4 \times 10^9$ bytes at $B_S = 1000$ GB/s.

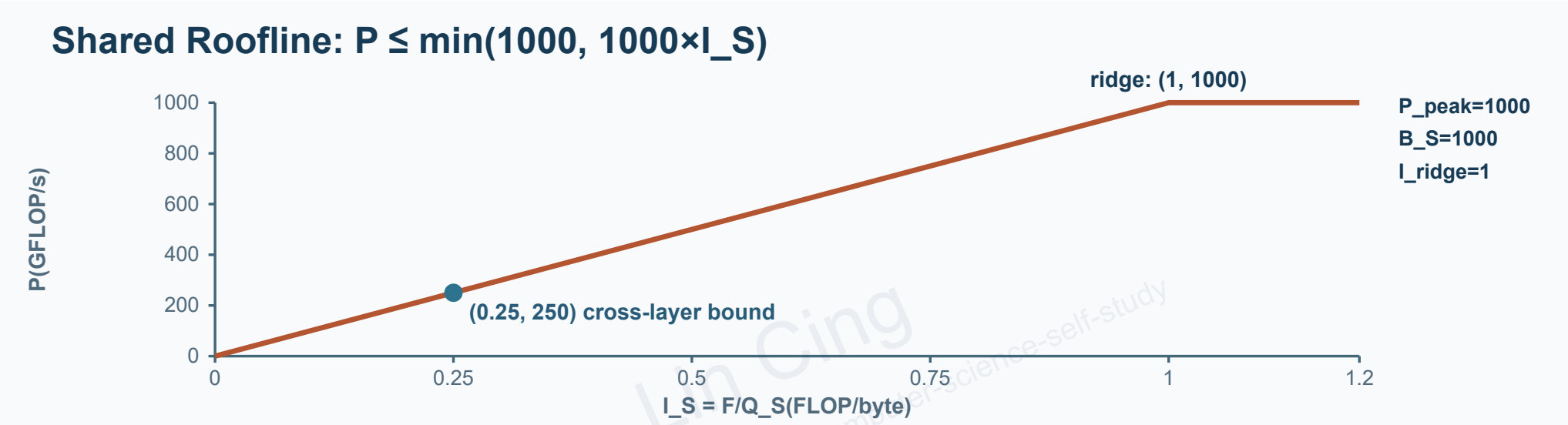
7.10 Deriving Roofline (continued)



$$I_D = 10^9/(0.25 \times 10^9) = 4, \qquad P_D \le \min(1000, 100(4)) = 400.$$

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.10 Deriving Roofline (continued)



$$I_S = 10^9/(4 \times 10^9) = 0.25, \qquad P_S \le \min(1000, 1000(0.25)) = 250.$$

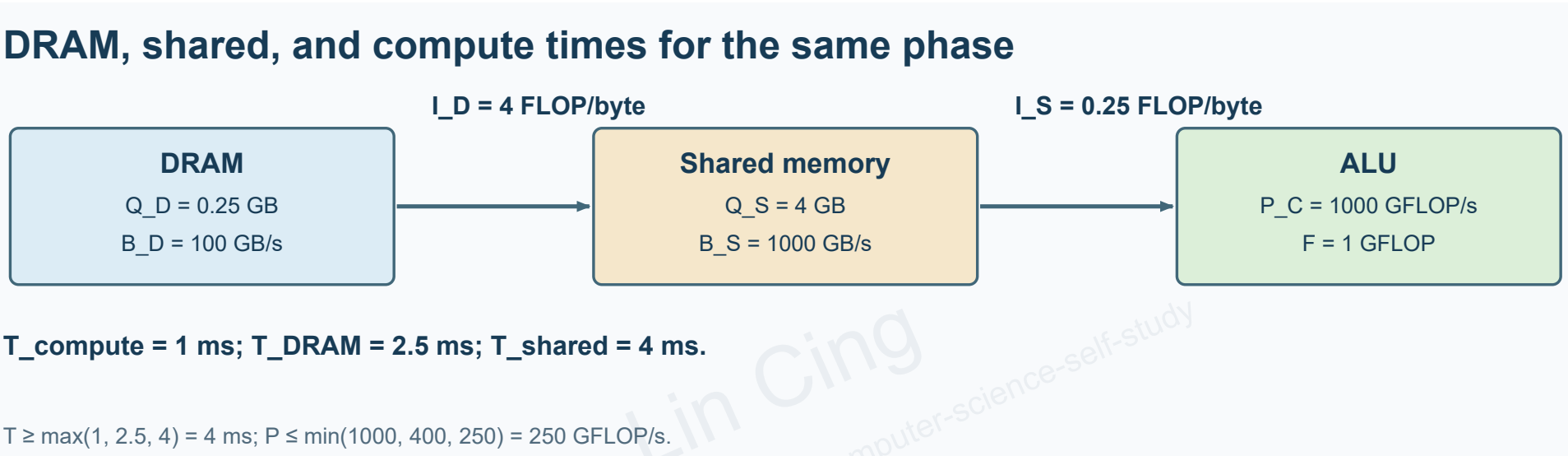
$$P \le \min(1000, 400, 250) = 250 \text{ GFLOP/s},$$

$$T \ge \max\left(\frac{10^9}{10^{12}}, \frac{0.25 \times 10^9}{100 \times 10^9}, \frac{4 \times 10^9}{1000 \times 10^9}\right) \text{ s} = \max(1, 2.5, 4) \text{ ms} = 4 \text{ ms}.$$

7.10 Deriving Roofline (continued)

DRAM's independent working point is(4, 400). The shared constraint lowers the cross-layer bound to(4, 250) when plotted using ID. Shared's own point is(0.25, 250), on its slope; its plateau remains 1000, not 250. These are model bounds, not measured rates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



A cache hit changes which interface carries bytes. Recompute intensity with that interface's bytes and bandwidth; never combine DRAM traffic with cache bandwidth. Separate roofs may be drawn separately, but the final same-phase bound takes the minimum across resources.

7.11 Historical multicore comparisons

The book compares four systems using sparse-matrix and structured-grid kernels. Their compute peaks, bandwidth, caches, TLBs, and programming environments differ. The distance from a measured point to its applicable ceiling can suggest a testable optimization opportunity.

Indirect sparse accesses and regular stencils can behave differently even at similar intensity because latency, compiler output, and access regularity differ. Historical results teach a method, not a current hardware ranking.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.12 Parallel fallacies

Weak scaling does not refute a fixed-work Amdahl bound. Peak FLOP/s does not determine the best machine for a memory-bound application. More threads can add oversubscription, cache pressure, synchronization, or power limits.

Coherence does not make racing software race-free. Correctness evidence and scaling evidence answer different questions.

Construct an execution trace or measured counterexample rather than relying on a slogan.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.13 Parallel-system synthesis

Hardware supplies resources; software decomposition, balance, communication, synchronization, and locality determine their useful work. Shared-memory and message-passing interfaces expose different costs.

Roofline bounds compute and bandwidth together. Measurements and counters test why a point lies below it. They do not turn a peak number into delivered application performance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

End-of-chapter check

Compute speedup with serial work and overhead. Distinguish strong from weak scaling. Explain SMP/NUMA/cluster addresses, compare thread scheduling, calculate graph cuts and hops, and derive Roofline bounds with consistent units.

Use the separate exercises and solutions.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

7.14 Historical perspective

Vector instructions amortize control over regular operations. Shared-memory multiprocessors extend load/store programming while requiring coherence and synchronization. Clusters scale resources through explicit messages and distinct failure boundaries.

Multicore, SMT, and GPUs exploit different task, thread, and data opportunities. None removes a serial critical path or makes remote communication local. Compare historical designs at fixed numerical validation, problem size, capacity, traffic, and power conditions.

In K16, every endpoint has fifteen direct links and a balanced cut has $8 \times 8 = 64$. A sixteen-port central crossbar normally gives each endpoint one switch attachment and has internal quadratic crosspoints. Do not combine their graph metrics.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, sections 7.2 and 7.8–7.10, printed pages 635–674 (physical PDF pages 662–701).

Williams, Waterman, and Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Communications of the ACM* 52(4), 2009, DOI.

The level-specific traffic convention follows the NERSC Roofline documentation. Each intensity uses bytes at its own interface. Numerical values here are declared model inputs and derived results.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

David A. Patterson, John L. Hennessy. Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study